

Reviewer Pack — Minimal Local Ring Norm Correlation with Communication Compl...

Entry cc842b0435e6 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-01 05:09:19 UTC

Minimal Local Ring Norm Correlation with Communication Complexity

Entry ID: cc842b0435e6 **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-01 05:09:19 UTC

1. Conjecture

Field A (mathematical branch): Local Ring Theory **Field B** (complexity object): Communication Complexity

Statement:

For every k -colorable graph G , the minimal local ring norm of its communication complexity is linearly correlated with its communication rank, such that $\text{min_ring_norm}(G) = \Theta(\text{communication_rank}(G))$.

Rationale (proposer's reasoning):

Local ring theory provides a framework to study arithmetic properties of polynomials in multiple variables. By mapping graph properties to polynomial representations, we can explore the relationship between local ring norms and communication complexity, which is a classical measure of efficiency in distributed computation.

Taxonomy category: communication_complexity (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 8c069c5aba45a407

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The correlation coefficient between $\text{min_ring_norm}(G)$ and $\text{communication_rank}(G)$ for a given k -colorable graph G is within ± 0.1 of its expected value.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	UNCERTAIN	HITS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: ADJACENT_OK against 1 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "local ring norm" AND "communication complexity" - "minimal local ring" AND communication rank - "k-colorable graph" AND communication rank AND local ring norm

Top relevant hits considered: - [<http://arxiv.org/abs/2003.08816v1>] Minimum Scan Cover with Angular Transition Costs

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_random_k_colorable_graph(n, k):
        graph = [[0] * n for _ in range(n)]
        colors = list(range(1, k + 1))

        for i in range(n):
            for j in range(i + 1, n):
                if random.choice([True, False]):
                    color = random.choice(colors)
                    graph[i][j] = color
                    graph[j][i] = color

        return graph

    def compute_min_ring_norm(graph):
        n = len(graph)
        min_norm = float('inf')

        for i in range(n):
            for j in range(i + 1, n):
                if graph[i][j]:
                    norm = abs(graph[i][j])
                    if norm < min_norm:
                        min_norm = norm

        return min_norm
```

```

def compute_communication_rank(graph):
    n = len(graph)
    rank = 0

    for i in range(n):
        for j in range(i + 1, n):
            if graph[i][j]:
                rank += 1

    return rank

n_max = 40
instances_tested = 30
min_ring_norms = []
communication_ranks = []

for _ in range(instances_tested):
    k = random.randint(2, 5)
    graph = generate_random_k_colorable_graph(n_max, k)

    min_ring_norm = compute_min_ring_norm(graph)
    communication_rank = compute_communication_rank(graph)

    min_ring_norms.append(min_ring_norm)
    communication_ranks.append(communication_rank)

correlation_coefficient = 0
if len(min_ring_norms) > 1 and len(communication_ranks) > 1:
    mean_min_ring_norm = sum(min_ring_norms) / len(min_ring_norms)
    mean_communication_rank = sum(communication_ranks) / len(communication_ranks)

    numerator = sum((min_ring_norm - mean_min_ring_norm) * (communication_rank - mean_communication_rank) for min_ring_norm, communication_rank in zip(min_ring_norms, communication_ranks))
    denominator = math.sqrt(sum((min_ring_norm - mean_min_ring_norm) ** 2 for min_ring_norm in min_ring_norms) + sum((communication_rank - mean_communication_rank) ** 2 for communication_rank in communication_ranks))

    correlation_coefficient = numerator / denominator

conjecture_holds = abs(correlation_coefficient) >= 0.9
counterexample = "" if conjecture_holds else f"Correlation coefficient {correlation_coefficient}"

return {
    "metric_name": "Correlation Coefficient",
    "metric_value": correlation_coefficient,
    "instances_tested": instances_tested,
    "n_max": n_max,
    "conjecture_holds": conjecture_holds,
    "counterexample": counterexample
}

if __name__ == "__main__":
    import sys

    seeds = [int(s) for s in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47]

    results = []
    for seed in seeds:
        result = run_trial(seed)

```

```

print(f"TRIAL: {result}")
results.append(result)

mean_metric_value = sum(result["metric_value"] for result in results) / len(results)
support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)

if all(result["conjecture_holds"] for result in results):
    print(f"RESULT: SUPPORTED mean={mean_metric_value:.4f} std=0.0000 support_fraction=1.0000")
elif support_fraction >= 0.8:
    print(f"RESULT: SUPPORTED mean={mean_metric_value:.4f} std=0.0000 support_fraction={support_fraction:.4f}")
else:
    first_failing_seed = next(result["seed"] for result in results if not result["conjecture_holds"])
    print(f"RESULT: FALSIFIED counterexample=\"Correlation coefficient outside  $\pm 0.1$ \" first_failing_seed={first_failing_seed}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_4e2273cc.py", line 102, in <module>
    result = run_trial(seed)
             ^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_4e2273cc.py", line 81, in run_trial
    correlation_coefficient = numerator / denominator
                             ~~~~~~^~~~~~
ZeroDivisionError: float division by zero

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed before producing data, which means that the correlation coefficient could not be calculated. Therefore, the pre-registered support condition was not unambiguously met. | next: Re-run the test with proper error handling to ensure it completes without crashing and produces the necessary data for calculating the correlation coefficient.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	15582
2	preregistration	ollama_remote	glm4:latest	0	0	16590
3	novelty	ollama_remote	glm4:latest	0	0	8156
4	novelty	ollama_remote	glm4:latest	0	0	13380
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	18446
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10161
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13192
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12379
9	judge	ollama_remote	glm4:latest	0	0	11532

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 119417 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/cc842b0435e6.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/cc842b0435e6.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/cc842b0435e6.tar.gz` (if generated)